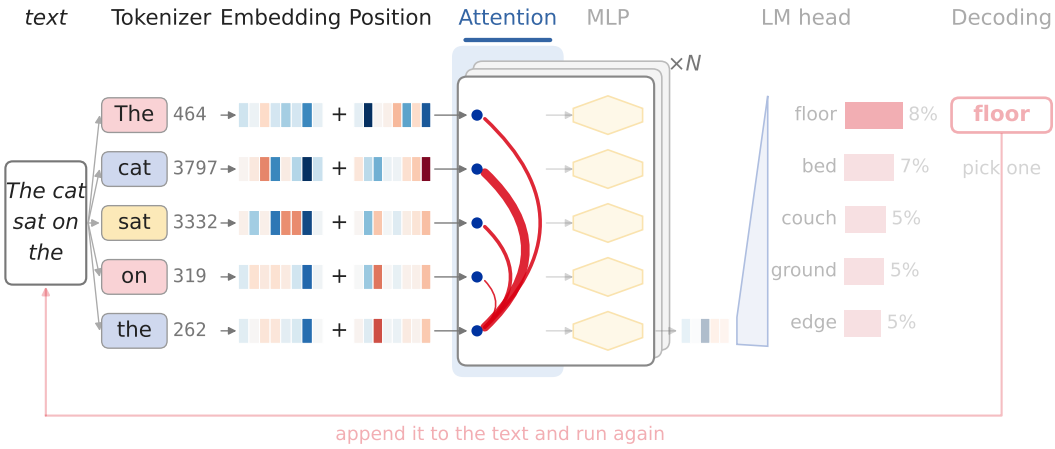
Attention

How tokens share meaning

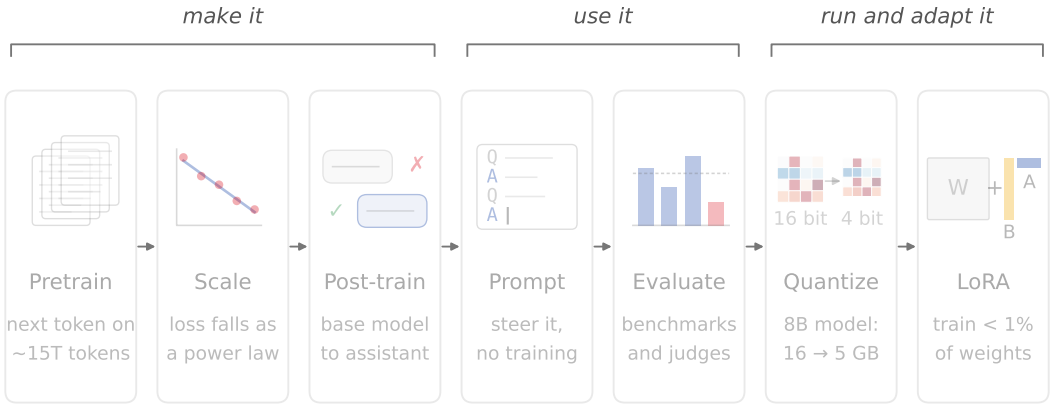
LLM chapter, session 3. Parts adapt LMU I2DL (CC BY 4.0) and 3Blue1Brown, “Deep Learning” chapters 5 and 6.

Where we are: one forward pass



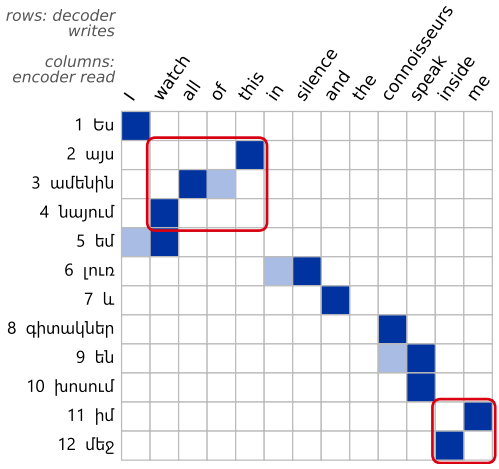
Today: the **attention** step - how each token's vector picks up meaning from the other tokens.

Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

[54] left two walls



Wall 1: one C for every step. The decoder writes the Armenian line one word per row. Each row needs one or two *different* English words, and seq2seq hands every row the same context vector C.

Wall 2: the queue. The RNN reads one token after another: 1,000 tokens are 1,000 steps in a row, while most of the GPU waits.

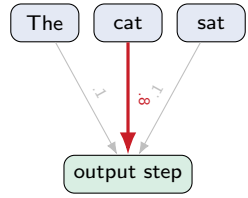
Predict: how would you fix wall 1? What should row 4 ("watch") get instead of C?

The fix, in one idea

Don't force everything through one vector. At **each** output step, let the decoder **look back over every input state** and take a *weighted average*: heavy weight on the words this step needs, light weight on the rest.

Row 4 ("watch") puts most of its weight on "watch"; row 5 picks a different mix. Every step gets its own context.

That weighted look-back *is* **attention**. Today unpacks this one sentence, and then takes it much further.



weights sum to 1; this step leans on "cat"

Outline

Wall 1: look back

Wall 2: drop the queue

One head, slowly

Wall 1: look back

A different context for every word you write.

Score, softmax, weighted sum

At each output step, attention does three things:

1. **Score** every input position: how relevant is it to this step?
2. **Softmax** the scores into weights that are positive and sum to 1.
3. **Weighted sum**: blend the input states by those weights. The blend is this step's own context, its personal C.

In translation the decoder's current state does the asking, and the encoder's states answer. The score itself: Bahdanau et al. fed the two states to a small learned network; later models just take their dot product, which is what we use today.

You have seen the three moves: LLM-2 ran them by hand on "dog bites man", and softmax and weighted sums are [41]. The new part is *what* gets scored and blended.

Learned, not drawn

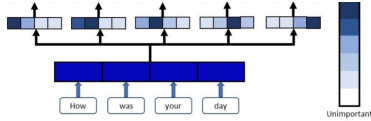


Figure: An illustration of a machine translation task using an encoder-decoder model with an attention mechanism. The attention weights at each time-step of the decoding/translation process indicate which parts of the input sequence are most relevant. There are 4 attention weights because there are 4 encoder states (Loye, 2019).



Translation: at each output step the model learns its own weights over the input words (the bar on the right).

(Bahdanau, Cho & Bengio, 2015: the first attention)

Captioning: weights over *pixels* while writing each word. (Xu et al., 2015)

Stack the steps and you get a grid like [54]'s, drawn by training instead of by hand (Bahdanau et al. show such grids for English to French). Wall 1 is down.

Source: LMU I2DL (slds-lmu), CC BY 4.0

Wall 2: drop the queue

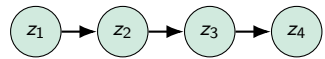
If every word can look at every word, who needs the recurrence?

The encoder still reads one word at a time

Attention took down wall 1: every output step gets its own mix of the encoder's states.

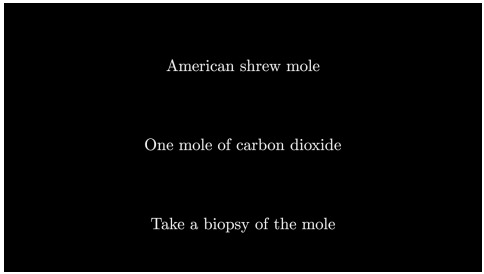
But those states are still built by an RNN, one after another: z_2 waits for z_1, z_3 waits for z_2. Wall 2 is still standing.

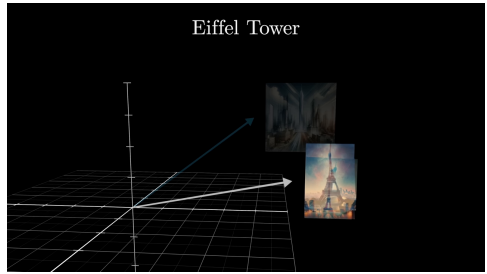
Predict: attention lets any position read any other position directly. Could it build the vectors themselves, with no recurrence at all?



each one waits for the one before

Words need their own sentence





"mole" means three different things, but the vector it starts with - its row of the table (LLM-2) - is identical in all three.

"tower", after "Eiffel", should move toward Paris, France, steel.

Same with "a machine learning **model**" and "a fashion **model**". Each word's vector has to change with its sentence, and what it needs is in that same sentence.

3Blue1Brown

"Each of you has what on the surface appears an unassailable alibi." she said, as the storm cleared and the crimson rays of dusk poured through the window where everyone was gathered. "But only one of you could have known about the loose screw on the window, while also having known where the second key was hidden. I am left, inescapably then, to the conclusion that therefore, the murderer was ???



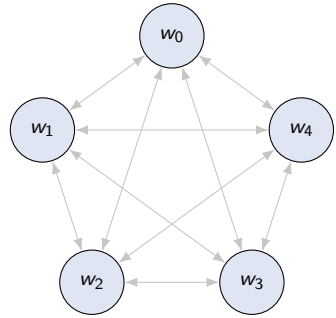
To fill the blank, the last vector must have soaked up the whole story.

The sentence talks to itself

In translation the *decoder's* state did the asking and the *encoder's* states answered: two sequences.

In **self**-attention the asking and the answering both come from the **same** sentence. Every word asks, and every word answers. One round is all-to-all messaging: each word gathers what it needs from all the others.

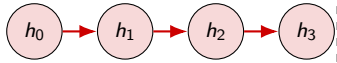
So yes to the predict: each word's new vector is built from all the others in one step, and the recurrence goes. The 2017 paper said it in its title: **"Attention Is All You Need."** (Vaswani et al., 2017)



every word attends to every word

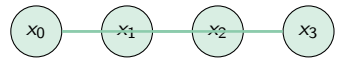
Sequential vs. all-to-all

RNN / LSTM



word 1 reaches word n after n steps

Self-attention



all-to-all: one step, direct

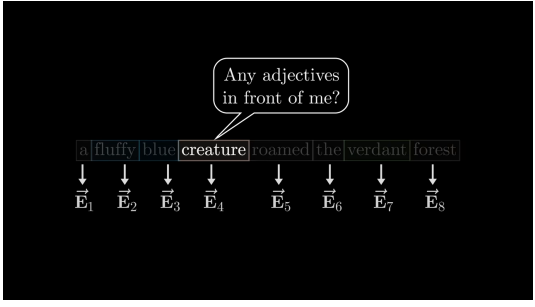
Three wins: **parallel** (all tokens at once) · **direct** (any token to any other) · **contextual** (each vector changes with its sentence). And it is all matrix products: the work a GPU is built for ([40], [54]'s second wall).

And word order? Self-attention alone cannot see it (LLM-2's "dog bites man"). That is why the position information goes into the vectors *before* the first attention layer.

One head, slowly

Every piece of the equation, and why it is there.

The goal, made concrete



3Blue1Brown

Sentence: *"a fluffy blue creature roamed the verdant forest."*

We want the noun **creature** to absorb its adjectives **fluffy** and **blue**, so its vector means *"a fluffy, blue creature."*

Two questions the mechanism must answer:

- Which words are **relevant** to creature?
- **What** should they hand over?

Predict first: who does “creature” listen to?

Toy vectors with 4 numbers, read as [adjective, fluffy, blue, noun]:

fluffy	$(1, 1, 0, 0)$
blue	$(1, 0, 1, 0)$
creature	$(0, 0, 0, 2)$

No learned matrices yet: each vector is its own query, its own key and its own value, the simplified version from LLM-2. Score = dot product divided by $\sqrt{4} = 2$; softmax by rows; weighted sum.

Predict: which of the three words gets the biggest share of creature’s attention?

Itself

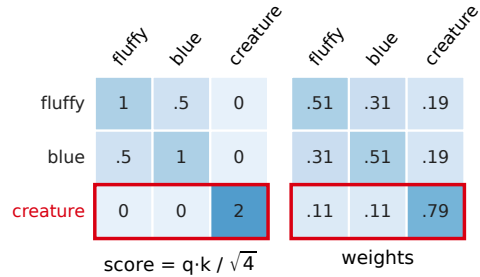
creature · creature $= 4$, so its own score is $4/2 = 2$; against fluffy and blue it scores 0.

Weights 0.107, 0.107, **0.787**: creature spends **79%** of its attention on itself. Its update:

$$0.107\,(1,1,0,0) + 0.107\,(1,0,1,0)$$

$$+\,0.787\,(0,0,0,2) = (0.21,\ 0.11,\ 0.11,\ 1.57)$$

Mostly more creature. And the grid is **symmetric**: fluffy looks at blue exactly as much as blue looks at fluffy.



Raw vectors ask "who is like me?", and the answer is always "me". Creature wants its adjectives.

A soft dictionary lookup

Think of looking something up:

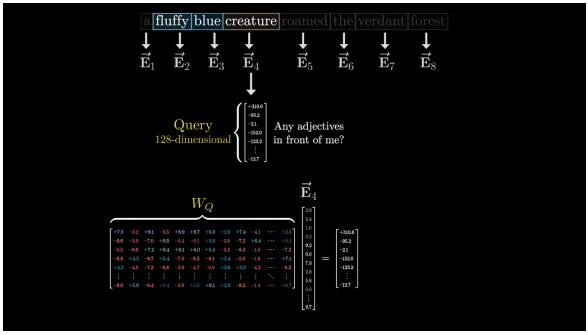
- You arrive with a **query**: what you want.
- Each entry has a **key**: the label it is filed under.
- Each entry has a **value**: its actual contents.

You compare your query with *every* key, then take the **values** of the good matches.

A normal dictionary is a **hard** lookup: one key matches exactly. Attention is a **soft** lookup: *every* key matches a little, weighted by similarity, and you blend all the values.

Every word makes its own query, key and value, each with its own learned matrix. That is how creature can stop asking "who is like me?".

Query: "what am I looking for?"



3Blue1Brown

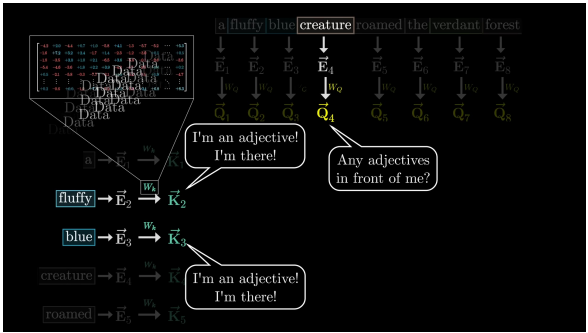
Multiply a word's vector by a learned matrix W_Q ($d_k \times d$, so 64×768) to get its **query**, a shorter vector; its length is called d_k (64 per head in GPT-2):

$$\mathbf{q}_i = W_Q \, \mathbf{e}_i.$$

Creature's query can encode the question *"any adjectives in front of me?"*

W_Q is **learned**: nobody writes "adjectives?" into it. Backprop from the next-token loss adjusts it like any other weight; the questions that survive are the ones that help predict.

Key: "what do I offer?"



3Blue1Brown

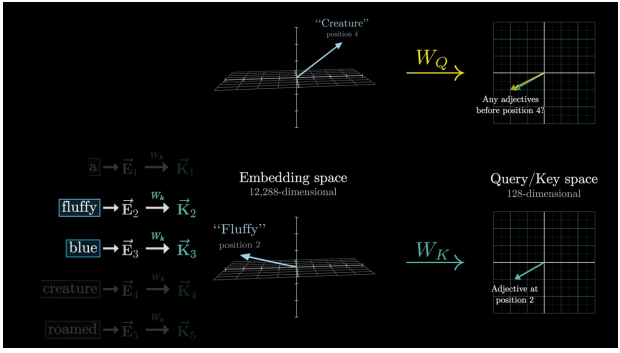
A second learned matrix W_K turns each vector into a **key**:

$$\mathbf{k}_j = W_K \, \mathbf{e}_j.$$

A key *answers* queries. Fluffy's key can encode *"I am an adjective, sitting right here."*

A key is a word *advertising* itself, so that the right queries can find it.

Queries and keys meet in one space



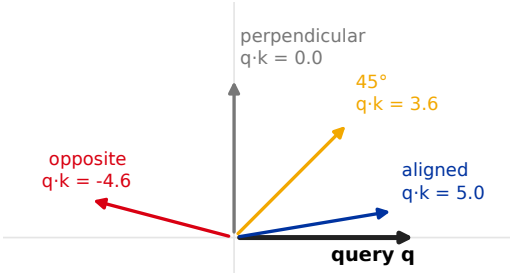
3Blue1Brown

W_Q and W_K **project** the big embedding space down to a shared, smaller **query/key space** (project: multiply by a matrix that makes the vector shorter, 768 numbers in, 64 out).

Good W_Q, W_K arrange it so that creature's query ("adjectives before me?") and fluffy's key ("adjective, here") point the **same way**.

That is all W_Q and W_K do: learn a space where "relevant" means "pointing the same way".

The dot product scores relevance



$\mathbf{q} \cdot \mathbf{k} = |\mathbf{q}|\,|\mathbf{k}| \cos\theta$ is **large** when the two point the same way, **zero** when they are perpendicular, and **negative** when they point apart.

So "how relevant is this key to my query?" collapses to **one number**.

All queries against all keys at once is one matrix product: $S = QK^\top$. Row i holds word i's scores against every word.

By hand: ask a different question

Make the query/key space as small as it gets: **one** number, a single axis that means "adjective match". (GPT-2's heads use 64 numbers: room for 64 kinds of question.)

- $w_Q = (0, 0, 0, 1)$: a **noun** asks along that axis, "any adjectives?". Queries: fluffy 0, blue 0, creature **2**.
- $w_K = (1, 0, 0, 0)$: an **adjective** answers on the same axis, "adjective here". Keys: fluffy **1**, blue **1**, creature 0.

Score = query $\times$ key, divided by $\sqrt{1} = 1$; then softmax by rows, as before.

Predict: what are creature's three weights now? And whom does fluffy listen to?

Creature finds its adjectives

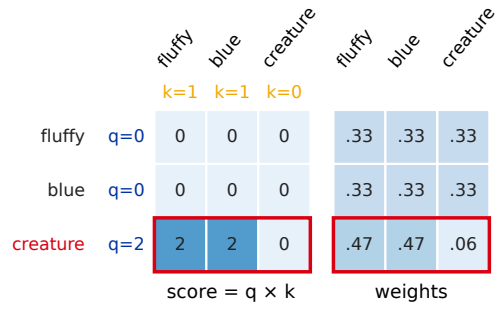
Creature's scores:
$2 \times 1,\ 2 \times 1,\ 2 \times 0 = 2,\ 2,\ 0$. Softmax:

$$\frac{(e^2,\ e^2,\ e^0)}{2e^2 + 1} = \frac{(7.39,\ 7.39,\ 1)}{15.78}$$

$$= (\mathbf{0.47},\ \mathbf{0.47},\ 0.06)$$

Fluffy's query is 0, so all its scores are 0: it asks for nothing and spreads evenly, 0.33 each.

Each **row** sums to 1, a word's attention budget; the columns do not.



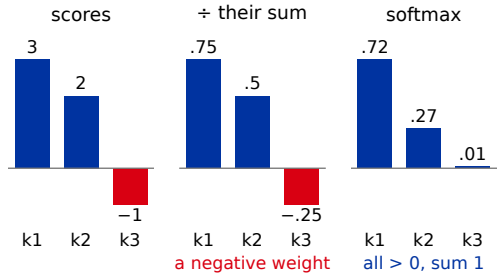
Creature now asks "any adjectives?" (79% on itself before, 6% now). And the grid is no longer symmetric: creature → fluffy scores 2, fluffy → creature scores 0.

Scores → weights: why softmax?

Raw scores can be any real number. But we want a **weighted average** of the values, so the weights must be

- non-negative, and
- sum to 1 (a distribution over "where to look").

Dividing by the sum fixes the second but not the first (middle). **Softmax** exponentiates first, so every weight is positive, then divides. It is also *soft*: it spreads weight over all the good keys instead of picking one, and it is smooth (differentiable), so we can train through it.

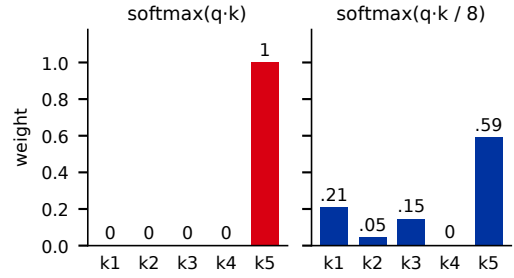


Why divide by $\sqrt{d_k}$?

A dot product over d_k numbers is a sum of d_k products. If the numbers have spread 1, each product has variance 1, and variances add: the sum has variance d_k, spread $\sqrt{d_k}$. For $d_k = 64$, about **8** (8.05 on 10,000 random pairs).

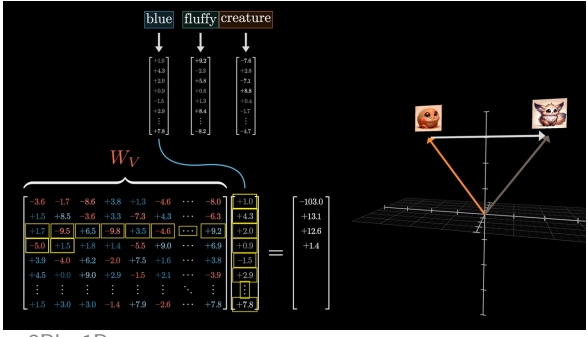
Feed scores that size to softmax and it **saturates**: one weight near 1, the rest near 0. At scores $(8, 0, 0)$ a loser gets 0.0003; raise its score by 1 and it gets 0.0009. At $(1, 0, 0)$ the same nudge moves it from 0.21 to 0.42. A nudge that changes nothing gives the gradient nothing to work with: **training stalls**.

Divide by the spread, $\sqrt{d_k} = 8$: back to about 1, softmax stays soft and trainable. Dividing by $d_k = 64$ would squash the scores near 0, and softmax would go flat.



Five random keys, $d_k = 64$: the same scores, before and after dividing by 8.

Value: “what do I hand over?”



3Blue1Brown

A third learned matrix W_V makes each word’s **value**, the information it hands over when attended to:

$$\mathbf{v}_j = W_V\,\mathbf{e}_j.$$

Fluffy’s value is, roughly, “the fluffiness to add to a noun”.

Key = how you are *found*; value = what you *give*. Different jobs, different matrices.

By hand: what gets handed over

W_V keeps fluffy, blue and noun and drops the "adjective" tag: values fluffy $(0, 1, 0, 0)$, blue $(0, 0, 1, 0)$, creature $(0, 0, 0, 2)$.

Creature's update, with its weights 0.47, 0.47, 0.06 to three digits:

$$0.468\,(0, 1, 0, 0) + 0.468\,(0, 0, 1, 0)$$

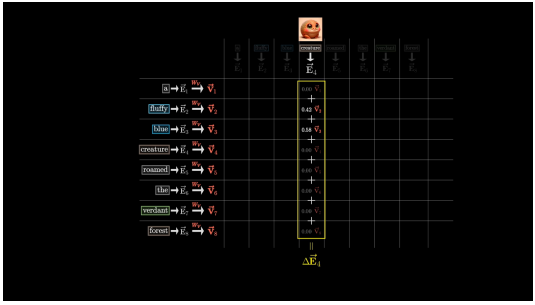
$$+\,0.063\,(0, 0, 0, 2) = (0,\ 0.47,\ 0.47,\ 0.13)$$

Added to creature itself (the residual connection, LLM-4): $(0,\ 0.47,\ 0.47,\ 2.13)$, a fluffy blue creature. With raw vectors it was $(0.21,\ 0.11,\ 0.11,\ 3.57)$.

	adjective	fluffy	blue	noun
creature, before	0	0	0	2
after, raw vectors	.21	.11	.11	3.57
after, learned W	0	.47	.47	2.13

Fluffy was *found* by its label ("adjective") and *handed over* its content (fluffiness). Found by one thing, give another: that is why $W_K \neq W_V$.

The output: values, weighted and summed



3Blue1Brown

In general: the weights say *how much* to listen, the values say *what* to pass. Word i's output is

$$\mathbf{o}_i = \sum_j \alpha_{ij}\, \mathbf{v}_j.$$

Every word is updated this way **at once**, in one parallel step. The output is **added** to the word's own vector, so words *refine* rather than overwrite (the residual connection, LLM-4).

Why three matrices?

One word plays **three roles** at once:

Query $\mathbf{q} = W_Q\mathbf{e}$	what I am *looking for*	creature: "any adjectives?"
Key $\mathbf{k} = W_K\mathbf{e}$	how I am *found*	fluffy: "adjective, here"
Value $\mathbf{v} = W_V\mathbf{e}$	what I *hand over*	fluffy: fluffiness, not the label

Without them, every word asks "who is like me?" and finds itself (79%), and A looks at B exactly as much as B looks at A. With them, the question, the label and the message are three separate things the model learns. They are the only weights in a single head (LLM-4 adds W^O when heads are combined).

Putting it together: the equation

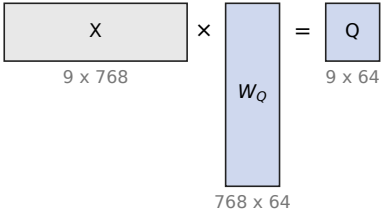
$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)\, V$$

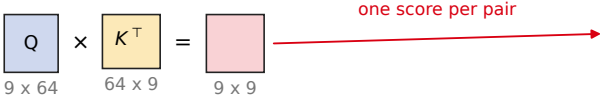
Read it left to right; it is exactly the three moves:

$QK^\top$	every query · every key = a grid of relevance **scores**
$\div\sqrt{d_k}$	keep the scores tame so softmax stays soft
$\mathrm{softmax}$	scores $\rightarrow$ **weights** (positive, sum to 1), row by row
$\times V$	**weighted sum** of the value vectors

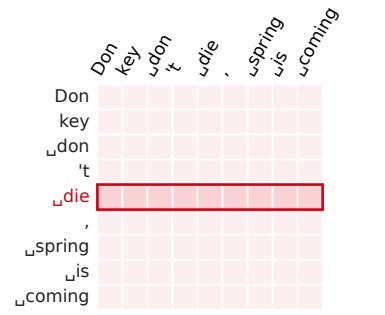
"Score, softmax, weighted sum": the same three moves as wall 1, now with learned Q, K, V from one sentence. In wall 1 the queries came from the decoder and the keys and values from the encoder: that version is **cross-attention** (LLM-5).

Shapes, on a real sentence





softmax each row, then multiply by V (9 x 64):
the output is 9 x 64. Boxes not to scale.



row = who asks, column = who answers;
the red row: what ␣die looks at

"Donkey don't die, spring is coming" is 6 words but 9 GPT-2 tokens (LLM-1): n counts tokens. The sizes 768 and 64 are GPT-2's.

The same shapes in GPT-2

- Each token is $d = 768$ numbers; a layer has 12 heads, each with $d_k = 64$.
- One head's W_Q, W_K, W_V are 64×768 each in our $W_Q\,\mathbf{e}$ notation: 147,456 numbers for the three. GPT-2's file stores them **transposed**, all twelve heads' in a single $768 \times 2{,}304$ matrix ($3 \times 12 \times 64 = 2{,}304$), because code multiplies rows (next slide).
- A full window of 1,024 tokens gives a $1{,}024 \times 1{,}024$ grid: 1,048,576 scores per head per layer, and 150,994,944 over 12 heads and 12 layers.
- Why 64 and not 768? $12 \times 64 = 768$: each head gets its own small space for its own kind of question (LLM-4).

The grid is $n \times n$: double the text and it gets **four times** bigger, while X, Q, K and V only double. That is the bill LLM-5 adds up.

Sizes read from GPT-2's config and weight file (local copy, 2026-10-05).

Attention in about 10 lines

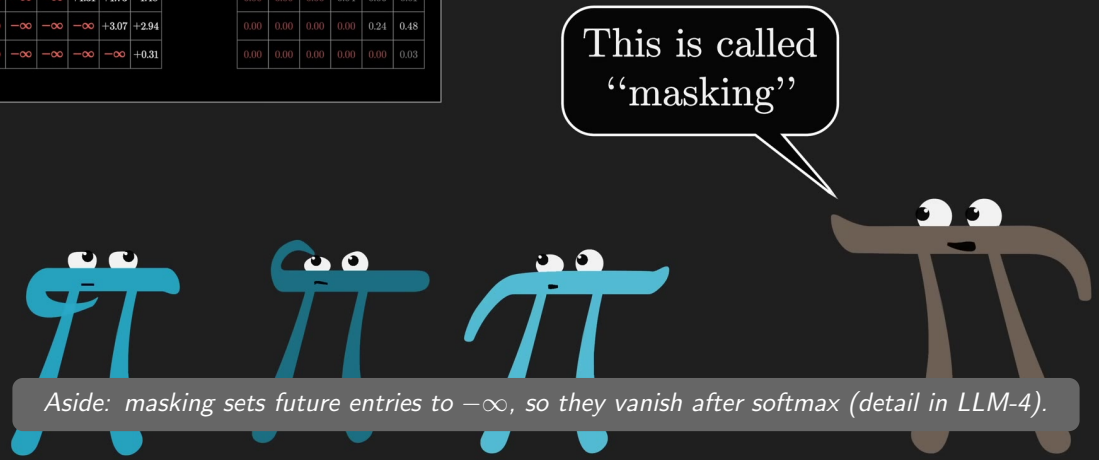
Every step is a matrix operation. In NumPy, with one token per **row** of `X`, so $\mathbf{q} = W_Q\,\mathbf{e}$ becomes `X @ Wq` with $\texttt{Wq} = W_Q^\top$ (768×64):

```
import numpy as np

def attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv        # three learned views
    d_k = Q.shape[1]
    scores = Q @ K.T / np.sqrt(d_k)          # relevance, scaled
    scores -= scores.max(1, keepdims=True)   # safe exp, same softmax
    weights = np.exp(scores)
    weights /= weights.sum(1, keepdims=True)  # softmax over each row
    return weights @ V                       # weighted sum of values
```

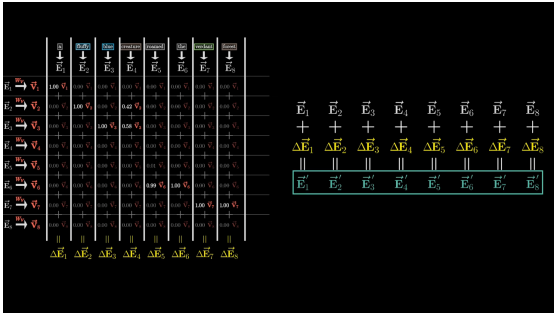
No loops over positions, no recurrence: three matrix products and a softmax. That simplicity is the whole point.

Unnormalized Attention Pattern							Normalized Attention Pattern					
+3.53	+0.80	+1.96	+4.48	+3.74	−1.95	softmax	1.00	0.75	0.69	0.92	0.46	0.00
−∞	−0.30	−0.21	+0.82	+0.29	+2.91		0.00	0.25	0.08	0.02	0.01	0.46
−∞	−∞	+0.89	+0.67	+2.99	−0.41		0.00	0.00	0.24	0.02	0.22	0.02
−∞	−∞	−∞	+1.31	+1.73	−1.48		0.00	0.00	0.00	0.04	0.06	0.01
−∞	−∞	−∞	−∞	+3.07	+2.94		0.00	0.00	0.00	0.00	0.24	0.48
−∞	−∞	−∞	−∞	−∞	+0.31		0.00	0.00	0.00	0.00	0.00	0.03



Aside: masking sets future entries to $-\infty$, so they vanish after softmax (detail in LLM-4).

One head, end to end



3Blue1Brown

- **Wall 1**, one C: every step gets its own weighted mix of the inputs.
- **Wall 2**, the queue: self-attention lets a sentence build its own vectors, all at once.
- Each word makes a query, a key and a value: ask, get found, hand over. Without them a word listens to itself.
- Score $QK^\top/\sqrt{d_k}$, softmax by rows, weighted sum of values; the grid is $n \times n$.

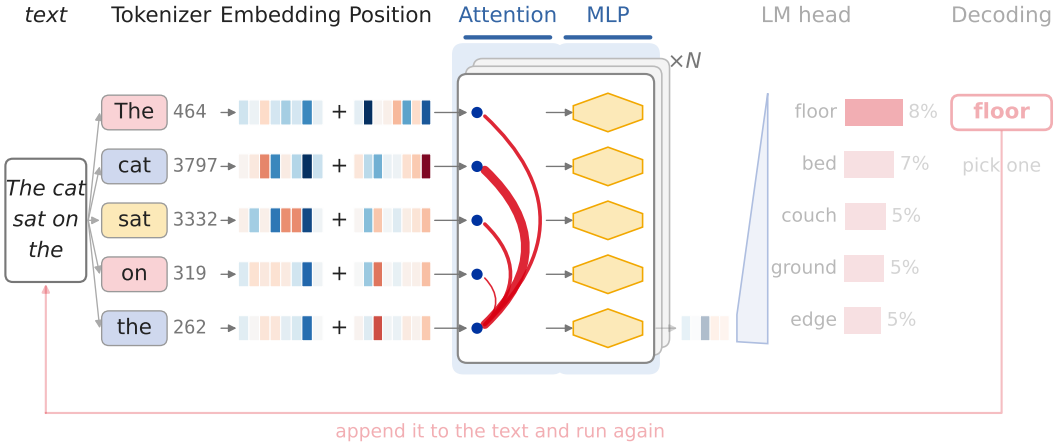
Next (LLM-4): many heads at once, and the block around them.

Try it yourself

Transformer Explainer (`poloclub.github.io/transformer-explainer`) runs GPT-2 small in your browser. Type your own sentence and look at the attention weights: the same $QK^\top$, softmax and $\times V$ as today, on a real model.

Real heads are messier than the creature toy, and different heads do different jobs. That is where LLM-4 starts.

Next on the map



Next (LLM-4): many heads at once, and the block around them.

Sources & further reading

- Bahdanau, Cho & Bengio (2015), *Neural Machine Translation by Jointly Learning to Align and Translate*: attention, for translation.
- Xu et al. (2015), *Show, Attend and Tell*: attention for image captioning.
- Vaswani et al. (2017), *Attention Is All You Need*: self-attention, the Transformer.
- Jay Alammar, *The Illustrated Transformer*; Andrej Karpathy, "Let's build GPT".
- Transformer Explainer: GPT-2 small in the browser, `poloclub.github.io/transformer-explainer`.
- **3Blue1Brown**, "Deep Learning" chapters 5 and 6: the visual intuition this deck leans on.

For the full, animated story of attention, watch 3Blue1Brown chapter 6. It is the clearest explanation out there.